

第三章

搜索探寻与问题求解

搜索算法在人工智能中具有基础性地位，其原因在于大量智能任务都可以抽象为“在众多可能方案中逐步作出选择”的过程。路径规划需要选择下一步移动方向，机器人操作需要决定下一时刻的控制动作，博弈程序需要判断当前局面的最优落子，复杂任务求解系统也需要决定先检索知识、先推理计算，还是先分解子任务。尽管这些任务在表面形式上差异显著，但其内部结构高度一致：系统处于某个状态，当前状态下存在若干可执行动作，动作会引起状态转移，而求解算法的目标是在庞大的可能性空间中找到一条通向目标状态的可行路径，甚至最优路径。

从这一视角出发，搜索的核心问题可以表述为：如何在有限计算资源约束下，有组织地探索状态空间，并尽快将计算资源集中到更有希望的分支上。为此，本节首先建立后续章节共用的概念框架，包括状态、动作、状态转移、目标测试、路径代价、搜索树、搜索图与剪枝等基本概念；随后介绍几类最基本的无信息搜索策略；最后分析搜索复杂性的来源，并说明本节内容与启发式搜索、对抗搜索以及蒙特卡洛树搜索之间的内在联系。

3.1 搜索算法基础

3.1.1 搜索的基本思想与直观意义 可以先从一个直观场景理解搜索问题。设校园中有一台送餐机器人，需要从食堂出发到达实验楼。机器人每到一个路口，都要在左转、右转、直行等可行动作中选择其一；每执行一次动作，机器人就会进入新的路口状态；当其最终到达实验楼门口时，问题便得到求解。若把机器人所处位置看作状态，把转向或前进看作动作，把每一段路的通行时间看作代价，那么这一任务就可以表示为一个标准的搜索问题。

再看一个更抽象的例子。八数码问题中的状态由棋盘上各数字块与空格的位置共同决定，动作是将空格向上、下、左、右方向移动。虽然它与路径规划在外观上差异很大，但同样可以表示为：从某个初始状态出发，经过一系列合法动作，抵达满足目标条件的状态。由此可见，搜索方法所关心的是问题的决策结构，而非问题的具体表象。

因此，搜索的直观本质可以概括为：面对大量可能的未来状态，算法需要有序地生成、评估并筛选潜在路径，从而逐步逼近目标。后续章节中出现的启发式搜索、对抗搜索和蒙特卡洛树搜索，都是在这一基本框架下，对“如何展开”“如何比较”“如何分配计算资源”所给出的不同回答。

3.1.2 搜索问题的形式化描述 为了用统一方式刻画不同问题，通常将搜索问题形式化为若干基本组成要素。

定义 3.1.1 (搜索问题) 一个搜索问题通常由状态空间 S 、初始状态 s_0 、动作集合 $A(s)$ 、状态转移函数 $\text{Result}(s, a)$ 、目标测试函数 $\text{Goal}(s)$ 以及单步代价函数 $c(s, a, s')$ 构成。搜索算法的任务是在这些要素定义的状态空间中，寻找一条从初始状态 s_0 到目标状态的可行路径；若问题存在代价约束，则进一步希望该路径具有最小总代价。

定义 3.1.2 (状态) 状态用于描述系统在某一时刻的完整情形。它应包含与后续决策有关的全部信息。在路径规划中，状态可以表示机器人当前位置；在八数码中，状态是当前棋盘布局；在博弈问题中，状态除了局面本身，还应包含“轮到哪一方行动”这一信息。

定义 3.1.3 (动作与状态转移) 动作表示在当前状态下允许执行的操作。对任意状态 s ，动作集合 $A(s)$ 给出了可执行动作的集合。状态转移函数 $\text{Result}(s, a)$ 描述了系统在状态 s 下执行动作 a 之后所达到的后继状态。动作回答“可以做什么”，状态转移回答“做完之后会到哪里”。

定义 3.1.4 (目标测试) 目标测试函数 $\text{Goal}(s)$ 用于判断状态 s 是否满足求解要求。该函数只负责判定“是否到达目标”，至于求得的路径是否为最优路径，则取决于路径代价定义以及具体搜索策略。

定义 3.1.5 (路径与路径代价) 设从初始状态 s_0 出发，依次执行动作

$$a_0, a_1, \dots, a_{k-1}$$

得到状态序列

$$s_0, s_1, \dots, s_k$$

其中

$$s_{i+1} = \text{Result}(s_i, a_i), \quad i = 0, 1, \dots, k-1$$

则这一状态序列对应一条路径，其总代价定义为

$$g(s_k) = \sum_{i=0}^{k-1} c(s_i, a_i, s_{i+1}) \quad (3.1)$$

路径代价是理解后续搜索算法的关键量。它体现了从起点到当前状态已经付出的真实代价。很多初学者容易把“步数最少”与“总代价最小”混为一谈，实际二者并不等价。设想机器人有两条可选路线，一条路径较短但道路拥堵，另一条路径较长但通行顺畅。若每走一步都记为单位代价，则前者步数更少；若以真实通行时间作为代价，则后者可能更优。因此，在一般搜索问题中，必须明确区分结点深度与路径代价。

进一步地，若某条路径既到达目标状态，又使总代价最小，则称其为最优解路径。后续 A* 搜索中的评价函数

$$f(n) = g(n) + h(n)$$

正是建立在“已付出的真实代价”与“未来代价估计”这两个量之上的。因而，本节对路径代价的理解将直接影响后续对启发式搜索的掌握。

3.1.3 一个贯穿全节的简单例子 为了帮助理解后续概念，下面给出一个统一的小例子。设某机器人需要从起点 A 出发，到达目标点 G 。图中每个字母表示一个位置，边上的数字表示从一个位置移动到相邻位置的代价。其形式化表示如表 3.1 所示。

表 3.1: 校园机器人路径搜索问题的形式化表示

要素	具体含义
状态空间 S	$\{A, B, C, D, E, F, G\}$ ，每个字母表示一个位置
初始状态 s_0	A
目标状态	G
动作集合 $A(s)$	在某位置可走向的相邻结点，例如在 A 处可走向 B 或 C
状态转移 $\text{Result}(s, a)$	执行动作后到达的新位置，例如从 A 走向 B 后到达状态 B
单步代价 $c(s, a, s')$	每条边的通行代价，例如 $c(A, a, B) = 1$ ， $c(A, a, C) = 4$
目标测试 $\text{Goal}(s)$	判断当前位置是否为 G

这一例子可以用如下简单代码表示：

```
graph = {
    "A": [("B", 1), ("C", 4)],
    "B": [("D", 2), ("E", 5)],
    "C": [("F", 1)],
    "D": [],
    "E": [("G", 1)],
    "F": [("G", 3)],
    "G": []
}
start = "A"
goal = "G"
```

其中， $(\text{"B"}, 1)$ 表示从当前结点可到达结点 B ，代价为 1。在这个例子中，从 A 到 G 的路径可以有多条，例如

$$A \rightarrow B \rightarrow E \rightarrow G$$

以及

$$A \rightarrow C \rightarrow F \rightarrow G$$

不同搜索算法的差异，就体现在它们优先扩展哪一条分支。

3.1.4 搜索树、搜索图与基本数据结构 当算法开始求解问题时，最自然的表示方式是搜索树。树中的每个结点表示“通过某条具体路径到达某个状态”的结果，根结点对应初始状态，从一个结点向下展开，就是枚举该状态下所有可能动作及其后继状态。搜索树能够清晰展示搜索过程中的父子关系、扩展顺序以及最终解路径，因此在算法教学中十分重要。

然而，在许多实际问题中，同一个状态可能通过不同路径重复到达。例如，机器人可以从路口 A 走到 B ，再从 B 绕回 A ；八数码也可能在不同移动序列之后回到同一棋盘布局。由此需要区分“结点”与“状态”两个概念。结点包含到达该状态的历史路径信息，状态只描述系统当前情形。不同结点可能对应同一个状态。

为了更准确地描述重复状态，人们引入搜索图的观点。若将搜索过程中所有相同状态进行合并，便得到一个状态图。树搜索按照路径展开，通常允许同一状态被多次生成；图搜索则会额外维护已访问状态信息，并在生成重复状态时进行检测与处理。对 A^* 算法而言，树搜索与图搜索在最优性条件上的差异，正是由这一点引出的。

在实际实现中，搜索算法通常会维护两个关键集合。第一个是边缘结点集合，常记为 OPEN 或 frontier，用于存放“已经生成但尚未扩展”的候选结点；第二个是已扩展集合，常记为 CLOSED 或 explored，用于存放“已经完成扩展”的结点。几乎所有经典搜索算法都可以视为在不断执行如下过程：从边缘集合中选出一个结点，检查

其是否满足目标条件；若尚未到达目标，则生成其后继结点并更新相关集合。

定义 3.1.6 (剪枝) 剪枝是指依据某种判据，提前终止对部分结点或分支的扩展，从而减少无效计算、压缩搜索空间。常见剪枝方式包括以下两类。其一是环路剪枝，即当当前路径回到祖先状态时，停止沿该分支继续展开；其二是重复状态剪枝，即当某一状态已经以更优路径代价到达时，舍弃当前较差路径。剪枝的核心作用是避免将计算资源浪费在明显无效或明显劣势的分支上。相关概念总结见表 3.2。

表 3.2: 搜索中几个容易混淆的基本概念

概念	含义	例子中的体现
状态	系统当前所处情形	当前位置是 $A, B, C, \dots, G$ 中的某一个
结点	搜索树中的记录单元	记录“通过某条路径到达 E ”
路径	从初始状态到当前状态的动作序列	$A \rightarrow B \rightarrow E$
路径代价	路径上各步代价之和	$g(E) = 1 + 5 = 6$
边缘结点	已生成但尚未扩展的结点集合	当前等待处理的候选路径
已扩展结点	已取出并生成后继的结点集合	已分析过其后继的结点

3.1.5 无信息搜索的基本策略 若搜索过程不借助任何额外领域知识，只依据问题本身给出的状态、动作和代价进行求解，则称为无信息搜索，也称盲目搜索。无信息搜索方法虽然较为基础，却极具教学价值，因为许多高级搜索算法都可以视为在无信息搜索框架中，引入了更有效的结点选择规则。

从统一形式看，许多搜索算法都遵循同样的基本流程。首先将初始结点加入边缘集合；然后重复执行以下步骤：从边缘集合中选取一个结点，若该结点满足目标测试，则返回解；否则扩展其后继结点，并将新生成结点按规则加入集合。不同搜索算法之间的关键差异，集中体现在“下一步优先扩展哪个结点”这一问题上。

广度优先搜索是一种总是优先扩展当前搜索树中较浅层结点的搜索策略。其基本思想是：从初始状态出发，首先扩展起点的所有直接后继；随后再依次扩展这些后继结点的后继，即按照从近到远、由浅入深的层次顺序逐层推进。只有当当前层的结点都已经被考察之后，算法才会继续进入下一层。因此，广度优先搜索体现出一种“先逐层展开，再继续深入”的搜索过程。

从数据结构角度看，广度优先搜索通常使用队列来维护边缘结点。新生成的结点按照产生的先后顺序进入队列，而扩展操作总是从队首取出最早进入的结点。正是由于队列具有先进先出的特点，算法才能保证搜索过程严格按照结点的深度顺序展开。与深度优先搜索倾向于沿单一路径不断深入不同，广度优先搜索更强调对同一层所有可能分支的均衡考察，因此在搜索早期能够较全面地覆盖离初始状态较近的区域。

广度优先搜索的主要优点在于完备性较强，并且在每一步动作代价相同的条件下具有最优性。也就是说，只要状态空间的分支因子有限且目标状态存在，广度优先搜

索就一定能够找到一个解；而且它找到的第一个解一定是步数最少的解。这一性质使其特别适用于求解无权图上的最短路径问题，或那些以“最少步数”作为目标的搜索任务。然而，该方法也存在明显局限：由于它必须同时保存某一层及下一层的大量候选结点，随着搜索深度增加，边缘结点数量往往迅速增长，从而导致较高的空间消耗和时间开销。在问题规模较大或解较深时，这种存储压力会变得尤为突出。

总体而言，广度优先搜索是一种完备性和最优性较强、但空间代价较高的基本搜索方法。它适合作为理解层次展开思想、最短路径搜索以及后续一致代价搜索、启发式搜索等方法的重要基础。

Algorithm 1 广度优先搜索

输入 初始状态 s_0 ，目标状态 g
输出 一条从 s_0 到 g 的路径，若不存在则返回失败

```
1: frontier  $\leftarrow$  只含初始路径  $[s_0]$  的队列
2: explored  $\leftarrow \emptyset$ 
3: while frontier 非空 do
4:   取出队首路径 path
5:   令  $s$  为 path 的最后一个状态
6:   if  $s = g$  then
7:     return path
8:   end if
9:   if  $s \notin \text{explored}$  then
10:    将  $s$  加入 explored
11:    for 每个后继状态  $s'$  属于  $\text{Successors}(s)$  do
12:      将新路径  $\text{path} + [s']$  加入队尾
13:    end for
14:   end if
15: end while
16: return 失败
```

深度优先搜索是一种总是优先沿当前分支向更深层结点继续扩展的搜索策略。其基本思想是：从初始状态出发，若当前结点尚未满足目标条件，则在其后继结点中选择一个继续向下搜索；只有当该分支无法继续扩展，或已经搜索失败时，算法才回溯到最近的分叉点，转而尝试其他尚未访问的分支。因此，深度优先搜索体现出一种“先走到底，再回头”的搜索过程。

从数据结构角度看，深度优先搜索通常使用栈来维护边缘结点，或者通过递归调用隐式地利用系统调用栈保存当前搜索路径。与广度优先搜索按层次逐步扩展不同，深度优先搜索更强调对单一路径的持续深入，这使得它在任意时刻通常只需要保存当前路径及少量尚未展开的兄弟结点，因此其空间开销相对较低。

深度优先搜索的主要优点在于实现简单、内存需求较小，特别适用于搜索空间较大但可用存储资源有限的场景。当目标结点恰好位于某条较深但较早被访问到的路径上时，深度优先搜索还可能较快找到一个可行解。然而，该方法也存在明显局限：由于

它优先深入当前分支，若状态空间中存在很深的无效路径、环路或大量无关分支，算法可能长时间停留在低质量区域，导致搜索效率较低；在无限深状态空间中，若不额外设置深度限制或去重机制，深度优先搜索甚至可能无法返回，从而不具备完备性

总体而言，深度优先搜索是一种空间效率较高但解质量与完备性较弱的基本搜索方法。它适合作为理解回溯思想、递归搜索以及后续深度限制搜索、迭代加深搜索等方法的重要基础。

Algorithm 2 深度优先搜索

输入 初始状态 s_0 ，目标状态 g

输出 一条从 s_0 到 g 的路径，若不存在则返回失败

```

1: frontier  $\leftarrow$  只含初始路径  $[s_0]$  的栈
2: explored  $\leftarrow \emptyset$ 
3: while frontier 非空 do
4:   取出栈顶路径 path
5:   令  $s$  为 path 的最后一个状态
6:   if  $s = g$  then
7:     return path
8:   end if
9:   if  $s \notin \text{explored}$  then
10:    将  $s$  加入 explored
11:    for 每个后继状态  $s'$  属于  $\text{Successors}(s)$  do
12:      将新路径  $\text{path} + [s']$  压入栈顶
13:    end for
14:   end if
15: end while
16: return 失败

```

一致代价搜索是一种按照累计路径代价最小原则选择扩展结点的无信息搜索策略。与广度优先搜索依据结点深度决定扩展顺序不同，一致代价搜索关注的是从初始状态到当前结点的实际总代价，即路径代价函数 $g(n)$ 。在每一步中，算法总是优先选择当前边缘集合中 $g(n)$ 最小的结点进行扩展，因此它本质上是在所有候选路径中不断优先推进“当前最便宜”的那一条。

从实现方式上看，一致代价搜索通常使用优先队列维护边缘结点，并以累计路径代价作为排序依据。每当一个结点被扩展时，算法会计算其各个后继结点的新累计代价，并将这些新结点重新插入优先队列中。由于扩展顺序由实际代价而非层数决定，因此当不同动作的代价不相同、某些路径虽然步数较多但总代价更低时，一致代价搜索比广度优先搜索更符合“寻找最优路径”的目标。

一致代价搜索的重要理论性质在于：只要所有单步代价均为正，该算法就是完备的，并且能够保证首次找到的目标结点对应一条总代价最小的最优路径。这一性质使它成为最优路径搜索中的经典基准方法。特别地，当所有边的代价都相等，且统一为 1 时，累计路径代价 $g(n)$ 实际上就等价于结点深度，因此一致代价搜索会退化为广度优

先搜索。从这个角度看，广度优先搜索可以被视为一致代价搜索在单位步代价条件下的特殊情形。

一致代价搜索的优点在于它能够严格依据真实代价进行决策，因此在代价不均匀的问题中具有明确的最优性保障；其不足则主要体现在搜索规模可能较大。由于算法必须维护一个按路径代价排序的优先队列，并且可能扩展大量看似“便宜”但并不通向目标的中间结点，因此其时间与空间消耗往往都比较高。在复杂度分析中，若所有步代价均不小于某个正数 $\varepsilon > 0$ ，最优解代价为 C^* ，则一致代价搜索需要扩展的结点数与 $\lfloor C^*/\varepsilon \rfloor$ 有关，复杂度通常可写成指数级形式。虽然其精确表达依赖具体代价分布，但总体上可认为该方法以更高的计算与存储代价换取了解的最优性保证。

Algorithm 3 一致代价搜索

```

输入 初始状态  $s_0$ ，目标状态  $g$ 
输出 代价最小的路径，若不存在则返回失败
1: frontier  $\leftarrow$  只含  $([s_0], 0)$  的优先队列
2: explored  $\leftarrow \emptyset$ 
3: while frontier 非空 do
4:   取出当前总代价最小的路径 path 及其代价 cost
5:   令  $s$  为 path 的最后一个状态
6:   if  $s = g$  then
7:     return path
8:   end if
9:   if  $s \notin \text{explored}$  then
10:    将  $s$  加入 explored
11:    for 每个后继状态  $s'$  及边代价  $w$  do
12:      将  $(\text{path} + [s'], \text{cost} + w)$  加入优先队列
13:    end for
14:   end if
15: end while
16: return 失败
  
```

从评价规则角度看，几类基本搜索策略之间存在清晰联系。广度优先搜索主要依据结点深度排序，一致代价搜索依据真实累计代价 $g(n)$ 排序，贪婪最佳优先搜索依据启发代价 $h(n)$ 排序，A* 搜索则综合考虑当前真实代价与未来估计代价，采用

$$f(n) = g(n) + h(n)$$

作为结点评价函数。这样理解之后，各类算法之间的关系会呈现出连续而统一的知识谱系。三种基本搜索策略的差异及其在同一例子中的扩展顺序，可分别由表 3.3 和表 3.4 概括。

表 3.3: 三种基本搜索策略的直观对比

算法	优先依据	适合解决的问题	在例子中的直观行为
广度优先搜索	结点深度最小	每一步代价相同的最短步数问题	先看离 A 最近的一层, 再逐层向外扩展
深度优先搜索	当前分支尽量向深处走	内存较紧张, 先快速找到一个可行解	先沿一条路径走到底, 再回头试其他路径
一致代价搜索	路径代价 $g(n)$ 最小	不同边代价不同的最优路径问题	每次都扩展当前总代价最小的路径

表 3.4: 同一例子下三种算法的扩展顺序示意

算法	可能的扩展顺序示意
广度优先搜索	$A \rightarrow B, C \rightarrow D, E, F \rightarrow G$
深度优先搜索	$A \rightarrow B \rightarrow D$, 回溯后再考察 E , 最后到达 G
一致代价搜索	先扩展总代价较小的路径, 例如优先比较 $A \rightarrow B$ 与 $A \rightarrow C$ 的累计代价

3.1.6 搜索复杂性与知识桥梁 搜索之所以具有挑战性, 根本原因在于状态空间的规模会随着搜索深度的增加而迅速膨胀。在许多问题中, 系统从一个状态出发, 往往可以通过若干不同动作转移到多个后继状态。若设每个状态平均有 b 个后继, 则称 b 为该问题的**分支因子**; 若目标结点位于深度 d , 则从初始状态开始, 在最坏情况下, 为了找到目标, 算法可能需要考察从第 0 层到第 d 层的全部结点。此时, 搜索树中最多包含的结点总数为

$$N(d) = 1 + b + b^2 + \cdots + b^d \quad (3.2)$$

当 $b > 1$ 时, 式 (3.2) 可进一步写为

$$N(d) = \frac{b^{d+1} - 1}{b - 1} \quad (3.3)$$

式 (3.3) 揭示了搜索问题最核心的计算难点。随着分支因子或解深度略有增加, 待处理结点数量就可能呈指数级增长, 这种现象通常称为**组合爆炸**。例如, 当 $b = 4, d = 10$ 时, 结点总数已超过百万量级; 当 $b = 10, d = 6$ 时, 结点总数同样会迅速达到百万量级。由此可见, 搜索算法设计的首要目标之一, 就是尽可能减少那些希望较小的分支扩展次数。从表 3.5 可以看出, 即使分支因子和深度只做小幅增加, 搜索规模也会迅速膨胀。

表 3.5: 分支因子与搜索深度对结点数量的影响

分支因子 b	深度 d	结点总数 $1 + b + b^2 + \cdots + b^d$
2	5	63
3	5	364
4	5	1365
4	10	1398101

基于上述分析，可以直观理解几类基础算法的复杂性来源。广度优先搜索为了保证不遗漏浅层解，通常需要同时保留大量边缘结点，因此其时间复杂度与空间复杂度都常写为

$$O(b^d)$$

深度优先搜索在最坏情况下可能一直搜索到最大深度 m ，因此时间复杂度通常写为

$$O(b^m)$$

而空间复杂度一般为

$$O(bm)$$

这说明在许多问题中，内存约束往往先于时间约束成为实际系统中的瓶颈。

评价一个搜索算法时，除了复杂性，还常关注两个核心性质。其一是**完备性**，即若解存在，算法最终能否找到解；其二是**最优性**，即算法找到的解是否一定具有最小总代价。后续启发式搜索将围绕这两个概念分析启发函数的可容性与一致性；对抗搜索会把“最优”从单智能体路径最优推广到多智能体零和博弈中的最优策略；蒙特卡洛树搜索则进一步考虑在计算资源有限条件下，如何通过采样与统计估计逐步逼近更优决策。

从更广泛的人工智能发展脉络来看，搜索并非只是早期人工智能中的经典主题，它至今仍深刻影响现代智能系统的设计。经典 A* 奠定了启发式最优路径搜索的理论基础；UCT 将多臂赌博机中的上置信界思想引入树搜索；AlphaGo 等系统则将神经网络评估与树搜索机制结合，使搜索具备了由学习模型引导的能力。因而，学习搜索算法基础的意义，在于建立一种统一的分析视角：面对庞大的可能性空间，智能系统始终需要回答同一个核心问题，即如何将有限计算资源优先投入到最值得探索的方向上。

3.2 启发式搜索

在第 3.1 节中, 我们已经讨论了广度优先搜索、深度优先搜索和一致代价搜索等基本策略。这些方法虽然重要, 但它们共同面对一个根本困难: 当状态空间迅速增大时, 单纯依靠系统枚举往往会消耗大量时间与存储资源。对于许多实际问题而言, 算法并非对未来一无所知。人们常常能够根据问题的结构, 给出某种“粗略但有用”的判断, 例如“离目标更近的城市通常更有希望”“棋盘上更接近胜势的局面更值得优先考虑”“机器人距离终点越近, 继续朝这个方向扩展通常越合理”。这类能够帮助搜索过程更早聚焦于有希望区域的信息, 统称为**启发信息**。

利用启发信息来指导结点扩展顺序的搜索方法, 被称为**有信息搜索** (informed search), 也称为**启发式搜索** (heuristic search)。与之相对, 像广度优先搜索和深度优先搜索这样仅依据状态转移规则展开探索、而不额外使用领域知识的方法, 则称为**无信息搜索** (uninformed search)。启发式搜索的核心目标, 并不是改变“搜索问题”的基本定义, 而是在相同的状态空间中, 借助对“目标距离”或“未来代价”的估计, 尽可能减少对无效分支的扩展。

从统一视角看, 许多搜索算法之间的主要差异, 其实都集中在一个问题上: **当 OPEN 表中有多个候选结点时, 应当优先扩展哪一个?** 广度优先搜索倾向于优先扩展深度较小的结点, 一致代价搜索倾向于优先扩展累计路径代价较小的结点, 而启发式搜索进一步引入对“未来还要付出多少代价”的估计。这样一来, 算法就不再只是机械地“按层”或“按已付代价”搜索, 而是试图朝更可能通向目标的方向推进。

本节将围绕两种最有代表性的启发式搜索方法展开。第一种是**贪婪最佳优先搜索** (greedy best-first search), 它只关心“当前结点看起来离目标有多近”; 第二种是**A* 搜索**, 它同时考虑“已经付出的真实代价”和“未来可能付出的估计代价”。前者直观、简洁、常常较快, 但不能保证找到最优解; 后者在合适条件下既完备又最优, 因此在路径规划、组合优化和智能决策中具有极其重要的地位。

3.2.1 启发函数与评价函数 为了用统一方式描述启发式搜索, 先引入两个最基本的函数: 启发函数和评价函数。

定义 3.2.1 (最优剩余代价) 设结点 n 表示某个搜索状态, 记从 n 出发到任一目标状态的**真实最小剩余代价**为

$$h^*(n) = \min_{\pi \in \Pi(n, \text{goal})} \text{Cost}(\pi) \quad (3.4)$$

其中 $\Pi(n, \text{goal})$ 表示从结点 n 到任一目标结点的全部可行路径集合, $\text{Cost}(\pi)$ 表示路径 π 的总代价。显然, $h^*(n)$ 描述了“从当前结点走到目标, 理论上至少还需要付出多少代价”。

定义 3.2.2 (启发函数) 启发函数 (heuristic function) 是对真实最小剩余代价

$h^*(n)$ 的估计，记为

$$h(n) \approx h^*(n), \quad h(n) \geq 0$$

(3.5)

它回答的问题是：“从当前结点 n 到目标，看起来大约还要花多少代价？”在地图路径规划中，常见的启发函数是当前位置到目标位置的直线距离；在八数码问题中，常见的启发函数是错位数字个数或曼哈顿距离之和。

定义 3.2.3 (路径代价函数) 记从初始结点 s_0 到当前结点 n 的已发生真实总代价为

$$g(n) = \sum_{i=0}^{k-1} c(s_i, a_i, s_{i+1})$$

(3.6)

其中路径为 $s_0 \rightarrow s_1 \rightarrow \cdots \rightarrow s_k = n$ 。函数 $g(n)$ 描述了算法到目前为止已经实际付出的代价。

定义 3.2.4 (评价函数) 评价函数 (evaluation function) 用于决定结点的扩展优先级，通常记为 $f(n)$ 。一般而言，评价函数值越小，表示该结点越值得优先扩展。

从这个角度看，启发式搜索的关键就在于：如何设计 $f(n)$ ，使它既能反映当前路径的真实成本，又能体现对未来搜索方向的合理判断。表 3.6 给出了几类典型算法在评价规则上的统一表达。

表 3.6: 常见搜索算法的统一评价视角

算法	优先依据	评价规则
广度优先搜索	结点深度最小	$f(n) = \text{depth}(n)$
一致代价搜索	已付真实代价最小	$f(n) = g(n)$
贪婪最佳优先搜索	预计离目标最近	$f(n) = h(n)$
A* 搜索	已付代价与未来估计之和最小	$f(n) = g(n) + h(n)$

从表 3.6 可以看出，A* 并不是凭空产生的一种全新思想，它可以看成是一致代价搜索与启发估计的自然结合。也正因如此，理解 $g(n)$ 与 $h(n)$ 的分工，是掌握整个启发式搜索框架的关键。简单说， $g(n)$ 反映“已经走了多少”， $h(n)$ 反映“估计还要走多少”，而 $f(n)$ 则试图回答“若继续沿当前方向前进，总体上看是否值得”。其关系如图 3.1 所示。

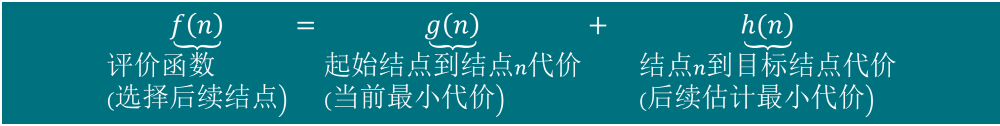


图 3.1: 评价函数、已付代价与启发代价之间的关系

为了帮助大家直观理解，可以从校园导航的例子出发。设机器人需要从宿舍楼前

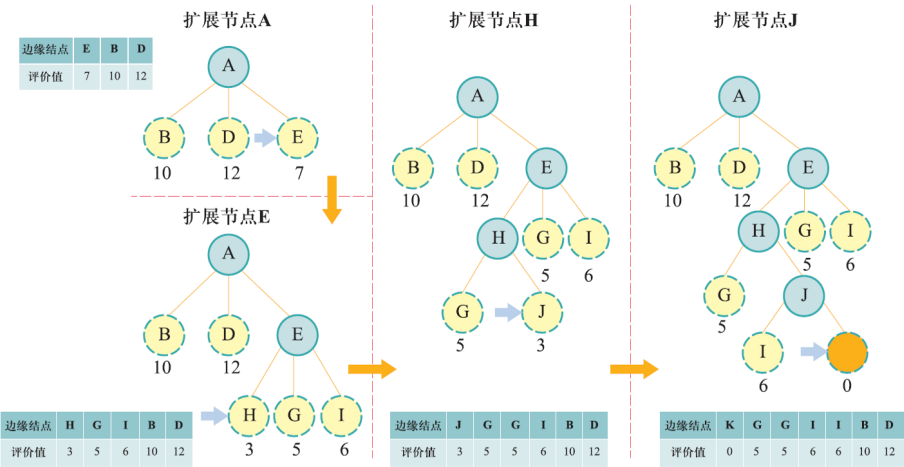


图 3.3: 贪婪最佳优先搜索过程示意图

表 3.7: 示例中部分结点的启发值与评价值

结点	$h(n)$	某条到达路径上的 $g(n)$	A^* 下的 $f(n) = g(n) + h(n)$
B	10	5	15
D	12	3	15
E	7	6	13
H	3	10	13
G	5	9	14
I	6	13	19
J	3	17	20
K	0	14 (经最优路径)	14

根据图 3.3 与表 3.7，算法从起点 A 出发，首先生成若干后继结点，例如 B 、 D 和 E ，其启发函数值分别为 10、12 和 7。由于 E 的 $h(E)$ 最小，所以算法优先扩展 E 。随后又生成了若干新结点，其中 H 的启发值更小，于是继续扩展 H 。如果继续按照“谁的 $h(n)$ 更小就优先扩展谁”的规则进行，算法就可能依次走向 J ，最终得到一条解路径

$$A \rightarrow E \rightarrow H \rightarrow J \rightarrow K$$

然而，这条路径虽然“看起来一直在逼近目标”，却并不是总代价最小的路径。实际上，更优的路径可能是

$$A \rightarrow E \rightarrow G \rightarrow K$$

其原因在于，贪婪最佳优先搜索只看 $h(n)$ ，却忽略了“为了到达当前结点已经付出了多少代价”。在示例中，结点 J 的启发值虽然较小，但到达它时的累计路径代价已经相当高；相反，结点 G 看起来离目标略远一些，可它对应的真实累计代价更小，整体上反而更优。

这恰好揭示了贪婪策略的典型特点：它擅长“迅速朝目标方向推进”，却不擅长“全局权衡已经付出的代价”。因此，贪婪最佳优先搜索常常在实践中表现得较快，尤其适用于“先尽快找到一个可行解”的场景，但它通常**不能保证最优性**。

下面给出贪婪最佳优先搜索的伪代码，如表 3.8 所示。

表 3.8: 贪婪最佳优先搜索伪代码

步骤	操作
1	将初始结点 s_0 放入 OPEN 表，并令 CLOSED 表为空。OPEN 按 $h(n)$ 的升序维护。
2	当 OPEN 表非空时，取出其中 $h(n)$ 最小的结点 n 。
3	若 n 满足目标测试，则返回从初始结点到 n 的路径。
4	将 n 加入 CLOSED 表，生成其所有合法后继结点。
5	对每个后继结点 n' ，若其未在 CLOSED 中出现，或需要按当前实现更新 OPEN，则将其插入 OPEN。
6	重复步骤 2 到步骤 5；若 OPEN 表为空仍未找到目标，则搜索失败。

从算法流程上看，贪婪最佳优先搜索与广度优先搜索、一致代价搜索的外形非常相似。它们都维护 OPEN 与 CLOSED 两个集合，都遵循“从 OPEN 中取一个最值得扩展的结点，检查是否为目标，再生成后继”的统一框架。真正不同的地方，只在于 OPEN 的排序准则发生了变化。正是这种“评价函数层面的变化”，使得算法的搜索行为产生了显著差异。

关于算法性质，需要给出更严谨的表述。在**有限状态空间**中，若采取环路检测或

重复状态处理机制，贪婪最佳优先搜索通常能够找到一个解；但它找到的解未必是最优解。其最坏情况下的时间复杂度与空间复杂度都可能达到指数级，常写为

$$O(b^m)$$

其中 b 是分支因子， m 是最大搜索深度。造成这一点的根本原因在于：即便启发函数能在多数情况下提供合理方向，只要估计不够准确，算法仍可能在大量分支上反复试探。

3.2.3 A* 搜索算法 贪婪最佳优先搜索的局限性启发我们：仅仅知道“离目标看起来还有多远”是不够的，还必须把“为了走到当前结点已经付出了多少代价”也纳入考虑。于是，自然得到如下评价函数：

$$f(n) = g(n) + h(n)$$

按照这个评价函数设计的搜索算法，就是著名的 **A* 搜索算法**。

其中， $g(n)$ 表示从初始结点到当前结点的真实累计代价， $h(n)$ 表示从当前结点到目标结点的估计剩余代价，可以理解为“经过当前结点到达目标所需总代价的估计值”。从这个意义上说，A* 每一步都在选择**估计总代价最小**的结点进行扩展。

A* 之所以重要，正在于它把“过去”和“未来”同时纳入评价。若只看 $g(n)$ ，就退化为一致代价搜索；若只看 $h(n)$ ，则退化为贪婪最佳优先搜索；而 A* 刚好位于两者之间，试图用一个统一的评价函数，把已经付出的真实代价与未来可能付出的估计代价综合起来。

这一点可以通过两个极端情形进一步理解：

当

$$h(n) = 0$$

对所有结点都成立时，A* 的评价函数退化为

$$f(n) = g(n)$$

此时 A* 就等价于一致代价搜索。

当

$$h(n) = h^*(n)$$

即启发函数恰好等于真实最小剩余代价时，A* 实际上拥有了“完美的未来信息”，此时它会高度聚焦于最优路径附近的结点，搜索效率达到理想状态。

图 3.4 给出了与前面相同问题上的 A* 搜索示意图。

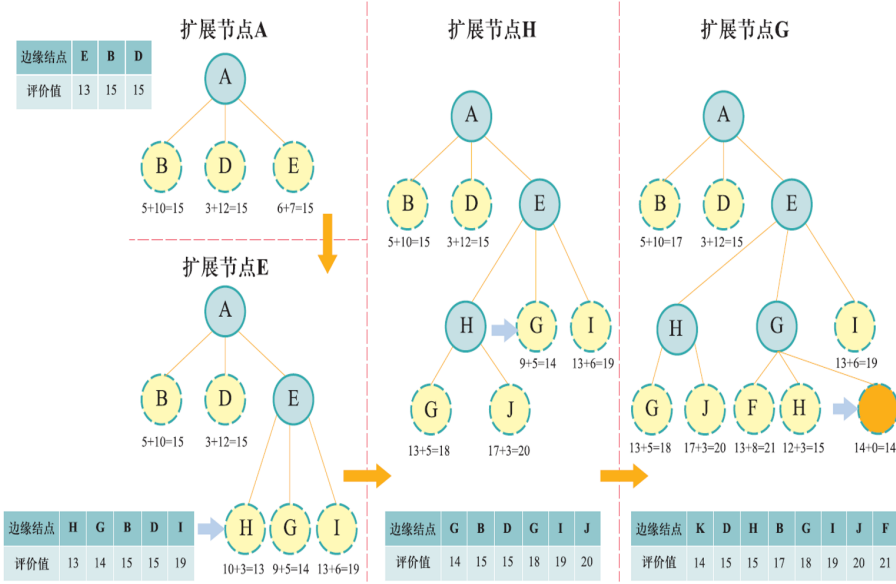


图 3.4: A* 搜索过程示意图

在表 3.7 的示例中，初始时生成 B 、 D 和 E 三个候选结点，其 A* 评价值分别为

$$f(B) = g(B) + h(B) = 5 + 10 = 15$$

$$f(D) = g(D) + h(D) = 3 + 12 = 15$$

$$f(E) = g(E) + h(E) = 6 + 7 = 13$$

因此，A* 首先扩展 E 。接着，若生成 H 、 G 和 I ，则它们的评价值分别为

$$f(H) = 10 + 3 = 13, \quad f(G) = 9 + 5 = 14, \quad f(I) = 13 + 6 = 19$$

到此为止，A* 的选择和贪婪最佳优先搜索可能暂时一致，因为 H 的估计总代价最小。但随着搜索继续推进，两者会出现关键分歧。若从 H 继续向下生成结点 J ，则

$$f(J) = g(J) + h(J) = 17 + 3 = 20$$

此时虽然 J 的启发值依旧很小，但它的累计代价已经偏大，所以 A* 不会像贪婪最佳优先搜索那样盲目追随 J ，而是转向扩展 G ，因为

$$f(G) = 14 < f(J) = 20$$

也正是这一步对“已付代价”的考量，使得 A^* 更有可能找到总代价最小的解。

因此， A^* 的核心优点可以概括为一句话：它不会因为某个结点“看起来离目标很近”就贸然选择它，而是同时审视“走到这里已经花了多少”与“从这里到目标大约还要多少”。

A^* 的伪代码如表 3.9 所示。这里采用图搜索版本的写法，并显式记录每个状态当前已知的最小 g 值，以便在发现更优路径时进行更新。

表 3.9: A^* 搜索伪代码

步骤	操作
1	初始化 OPEN 表与 CLOSED 表。将初始结点 s_0 放入 OPEN，并设置 $g(s_0) = 0, f(s_0) = h(s_0)$ 。OPEN 按 $f(n)$ 升序维护。
2	当 OPEN 表非空时，取出其中 $f(n)$ 最小的结点 n 。
3	若 n 满足目标测试，则返回从初始结点到 n 的路径。
4	将 n 加入 CLOSED 表，并生成其所有后继结点 n' 。
5	对每个后继结点 n' ，计算候选路径代价 $t = g(n) + c(n, a, n')$ 。
6	若 n' 尚未出现过，或 $t < g(n')$ ，则更新其前驱为 n ，并令 $g(n') = t, \quad f(n') = g(n') + h(n').$ 若有需要，则将 n' 插入 OPEN 或重新放回 OPEN。
7	重复步骤 2 到步骤 6；若 OPEN 为空仍未找到目标，则搜索失败。

从教学上看， A^* 的意义远不止于“一个更好的算法”。更重要的是，它为后续许多智能决策方法建立了基本范式：评价一个候选动作时，既要看当前已经观察到的真实收益，也要估计未来可能带来的影响。这种思路在路径规划、调度优化、博弈搜索，乃至今天很多神经搜索框架中都反复出现。

3.2.4 A^* 算法中启发函数的性质 A^* 能否真正发挥作用，并不只取决于算法流程本身，还高度依赖于启发函数 $h(n)$ 的性质。一个设计得好的启发函数，能够显著减少搜索空间；一个设计得差的启发函数，则可能让 A^* 退化为几乎盲目的搜索。因此，有必要对启发函数提出更严格的理论要求。

定义 3.2.5（可容性） 若对任意结点 n ，启发函数满足

$$0 \leq h(n) \leq h^*(n) \tag{3.8}$$

且对任意目标结点 n_g 有

$$h(n_g) = 0 \tag{3.9}$$

则称 $h(n)$ 是**可容的** (admissible)。

可容性的直观含义是：**启发函数不能高估从当前结点到目标结点的真实最小剩余代价**。换句话说，它可以“保守”，可以“低估”，甚至可以非常粗略，但不能“吹得太大”。例如在地图导航中，若把两点之间的直线距离乘以单位速度所需时间作为启发值，那么这个估计通常不会超过真实道路距离对应的通行时间，因此往往是可容的。

定义 3.2.6 (一致性) 若对任意结点 n 及其任一后继结点 n' ，启发函数满足

$$h(n) \leq c(n, a, n') + h(n') \quad (3.10)$$

则称 $h(n)$ 是**一致的** (consistent)，也称**单调的** (monotone)。

一致性的形式与三角不等式非常相似。它要求从当前结点对目标的估计，不能大于“先走一步到后继结点的真实代价，再加上从后继结点到目标的估计代价”。这意味着启发函数在相邻状态之间变化得不能过于剧烈，否则就会破坏 A* 在图搜索中的理论保证。

为了更直观地理解一致性，可以把它解释成一种“局部合理性”约束：如果从 n 走一步就能到达 n' ，那么对目标距离的估计应当随着这一步而平滑变化，而不应出现前后严重矛盾的情况。图 3.5 给出了这一关系的示意。

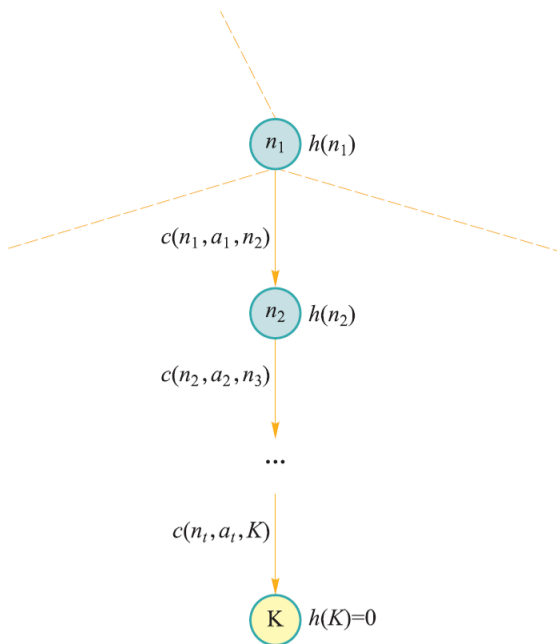


图 3.5: 启发函数一致性的几何直观示意图

一致性比可容性更强。下面给出严格证明。

定理 3.2.1 若启发函数 $h(n)$ 一致，则它一定可容。

证明：设从结点 $n = n_0$ 到某个目标结点 n_k 的一条最优路径为

$$n_0 \rightarrow n_1 \rightarrow \cdots \rightarrow n_k,$$

其中 n_k 为目标结点，因此

$$h(n_k) = 0.$$

由一致性可知，对路径上的任意相邻结点，都有

$$h(n_i) \leq c(n_i, a_i, n_{i+1}) + h(n_{i+1}), \quad i = 0, 1, \dots, k-1.$$

依次展开可得

$$\begin{aligned} h(n_0) &\leq c(n_0, a_0, n_1) + h(n_1) \\ &\leq c(n_0, a_0, n_1) + c(n_1, a_1, n_2) + h(n_2) \\ &\leq \cdots \\ &\leq \sum_{i=0}^{k-1} c(n_i, a_i, n_{i+1}) + h(n_k). \end{aligned}$$

由于 $h(n_k) = 0$ ，于是

$$h(n_0) \leq \sum_{i=0}^{k-1} c(n_i, a_i, n_{i+1}) = h^*(n_0).$$

又由启发函数通常取非负值，故

$$0 \leq h(n_0) \leq h^*(n_0).$$

这正是可容性的定义，因此一致性蕴含可容性。证毕。

表 3.10 对这两个概念作进一步比较。

表 3.10: 可容性与一致性的比较

性质	数学条件	主要作用
可容性	$0 \leq h(n) \leq h^*(n)$	保证树搜索版本 A^* 的最优性
一致性	$h(n) \leq c(n, a, n') + h(n')$	保证图搜索版本 A^* 的最优性与 f 值单调性
关系	一致性 $\Rightarrow$ 可容性	一致性更强

除了最优性分析，一致性还有一个非常重要的推论。若 $h(n)$ 一致，则沿任意

路径有

$$f(n') = g(n') + h(n') \geq g(n) + h(n) = f(n) \quad (3.11)$$

也就是说， A^* 中的 f 值沿路径是非递减的。这一性质对图搜索尤其关键，因为它意味着一个结点一旦以最小 f 值从 OPEN 中弹出，其对应的最优路径代价已经得到了稳定保证，从而为 CLOSED 表的正确使用奠定了基础。

3.2.5 A^* 算法性能分析 评价 A^* 的性能时，通常重点关注四个方面：完备性、最优性、时间复杂度和空间复杂度。

先看完备性。

定理 3.2.2 若搜索问题满足以下条件，则 A^* 是完备的：

- (1) 每个结点的分支数有限；
- (2) 每一步动作代价都存在正下界，即存在 $\varepsilon > 0$ ，使得

$$c(n, a, n') \geq \varepsilon$$

- (3) 启发函数有下界，通常可取非负。

上述条件的直观含义是：搜索树不会在单层无限分叉，路径代价不会因为无限多个“几乎不花代价”的动作而陷入病态循环，评价函数也不会向负无穷方向失控。在这些前提下， A^* 不会永远卡在某个有限代价区间内反复扩展无穷多个结点，因此只要解存在，就最终能够找到解。

再看最优性。 A^* 的最优性分析需要区分**树搜索**和**图搜索**两种情形。

定理 3.2.3 在树搜索中，若启发函数 $h(n)$ 可容，则 A^* 一定能找到最优解。

证明：设最优目标结点的总代价为 C^* 。反设 A^* 第一个取出的目标结点 G 不是最优的，其路径代价为

$$g(G) > C^*$$

由于目标结点满足 $h(G) = 0$ ，故

$$f(G) = g(G) > C^*$$

考虑一条最优路径从初始结点到最优目标结点的搜索过程。在 A^* 取出 G 之前，这条最优路径上必然存在至少一个尚未被扩展的结点 n 位于 OPEN 表中。对该结点，由于启发函数可容，有

$$h(n) \leq h^*(n)$$

于是

$$f(n) = g(n) + h(n) \leq g(n) + h^*(n) = C^*$$

这说明 OPEN 表中至少存在一个结点 n 使得

$$f(n) \leq C^* < f(G)$$

但 A^* 总是优先取出 f 值最小的结点，因此它不可能先取出 G 再去处理 n 。矛盾。故假设不成立，第一个被取出的目标结点必为最优解。证毕。

这个证明非常重要。它揭示了 A^* 最优性的核心逻辑：**只要启发函数不高估未来代价，那么最优路径上的某个前沿结点，其 f 值就永远不会超过最优解代价。于是，任何总代价更大的目标结点都不可能抢在它前面被扩展。**

然而，对于图搜索，情况更复杂。因为图搜索会把已经扩展过的状态放入 CLOSED 表，而同一个状态可能由不同路径多次到达。如果第一次到达某状态的路径并非最优，而算法又过早地把它“封死”，那么后续更优路径就可能失去机会。

因此，在图搜索中，仅有可容性还不够。此时需要更强的条件。

定理 3.2.4 在图搜索中，若启发函数一致，则 A^* 能保证最优性。

其根本原因在于，一致性保证了沿路径的 f 值单调不减，从而保证一个结点第一次以最小 f 值从 OPEN 中取出时，对应的 g 值已经是最优的。这样 CLOSED 表对该状态的封存才是安全的。

若启发函数仅可容而不一致，则图搜索版 A^* 可能失去最优性。应对这一问题有两种常见办法：

第一，要求启发函数满足一致性；

第二，在实现上允许“重新打开”结点，即当发现某个已访问状态有更优的 g 值时，更新其前驱并将其重新插入 OPEN 表。

图 3.6 给出了图搜索中“发现更优路径后重新更新前驱”的示意图。

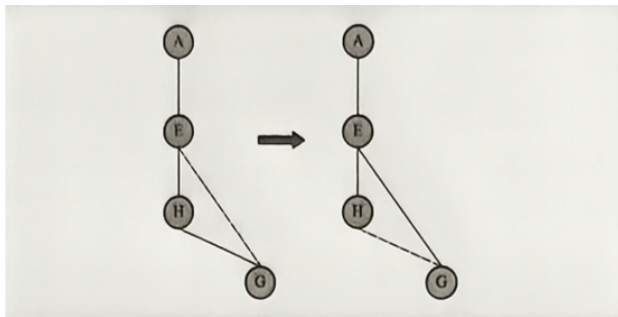


图 3.6: 图搜索中发现更优路径时的状态更新示意图

除了完备性与最优性， A^* 的复杂性也值得强调。尽管它通常比无信息搜索高效得多，但在最坏情况下， A^* 的时间复杂度与空间复杂度仍可能是指数级的。若启发函数过于粗糙，例如

$$h(n) \approx 0$$

则 A^* 会接近一致代价搜索，几乎失去启发式优势；反之，若启发函数更接近真实最优剩余代价 $h^*(n)$ ，则 A^* 往往能显著减少扩展结点数。

这一点可以进一步形式化为“启发函数支配性”的概念。设 $h_1(n)$ 与 $h_2(n)$ 都是可容启发函数，若对所有结点都有

$$h_1(n) \leq h_2(n) \leq h^*(n)$$

则称 h_2 支配 h_1 。一般来说，在其他条件相同的情况下，使用 h_2 的 A^* 不会比使用 h_1 扩展更多结点。这说明：**更接近真实代价、同时又不高估的启发函数，通常更有搜索效率。**

为了帮助大家建立整体认识，表 3.11 对贪婪最佳优先搜索与 A^* 作最后比较。

表 3.11: 贪婪最佳优先搜索与 A^* 的比较

比较维度	贪婪最佳优先搜索	A^* 搜索
评价函数	$f(n) = h(n)$	$f(n) = g(n) + h(n)$
关注重点	只看未来估计	同时看已付代价与未来估计
搜索行为	更激进，更像“朝目标冲”	更稳健，更像“综合权衡后前进”
是否一定最优	否	条件满足时是
典型优点	往往较快找到一个解	在理论与实践上兼顾效率与最优性
典型风险	容易陷入局部上看起来最优的方向	启发函数差时仍可能代价较高

综上，启发式搜索的思想可以概括为：**借助对未来的合理估计，把有限的计算资源优先投向更有希望的区域。**贪婪最佳优先搜索体现了“快速逼近目标”的直观策略， A^* 则进一步把“已经付出的真实代价”纳入权衡，形成了既有理论保证、又有广泛应用价值的经典算法框架。理解这一框架，对于后续学习对抗搜索中的估值传播、以及蒙特卡洛树搜索中的探索与利用平衡，都具有十分重要的桥梁作用。

3.3 对抗搜索

在前面的搜索问题中，环境通常被看作静态的、确定的，智能体只需要考虑“我下一步该怎样走”。然而在博弈问题中，情况发生了根本变化：智能体面对的不是一个被动环境，而是一个会主动反制的对手。也就是说，当前动作的好坏，不仅取决于它能把局面带到哪里，还取决于对手随后会怎样回应。因此，对抗搜索的核心任务，不再只是寻找一条从初始状态到目标状态的最短路径，而是要在“我方行动”与“对手反制”的交替过程中，求得对自己最有利的策略。

从本科教学的角度看，对抗搜索非常适合作为“单智能体搜索”向“多智能体决

策”过渡的桥梁。它保留了搜索树、状态、动作、终止条件等基本概念，同时又引入了一个新的关键因素，即同一个结点的价值，不再只由当前路径代价决定，而要由后续双方的最优博弈共同决定。正因为如此，对抗搜索中的“最优”也有了新的含义：它不表示路径最短，而表示在假设双方都理性行动时，我方能够保证得到的最好结果。

本节主要讨论最经典的一类情形：完全信息、确定性、轮流行动、二人零和博弈。在这一设定下，最基本的求解方法是 Minimax 搜索，而在搜索树规模较大时，还需要借助 Alpha Beta 剪枝来减少无效扩展。为了便于理解，我们先从形式化定义入手，再逐步解释算法思想、递推公式和复杂性分析。

3.3.1 对抗搜索的基本思想与形式化描述 在二人零和博弈中，一方的收益增加，另一方的收益就会等量减少，因此双方利益完全对立。典型例子包括井字棋、五子棋、国际象棋以及许多棋盘类策略问题。虽然这些任务外观不同，但都可以抽象成同一类搜索模型。

表 3.12: 对抗搜索问题的典型假设

假设	含义
完全信息	双方都能看到当前完整局面，不存在隐藏信息
确定性	执行动作后的结果确定，不包含随机掷骰子等机制
轮流行动	任意时刻只有一方行动，双方交替决策
二人零和	一方收益增加时，另一方收益等量减少，双方收益和恒为 0
理性决策	双方都试图选择对自己最有利的动作

这类问题通常满足表 3.12 所示的典型假设。在上述假设下，一个对抗搜索问题可以形式化表示为

$$\mathcal{G} = \langle S, s_0, A, \text{Player}, \text{Result}, \text{Terminal}, U \rangle$$

其中各组成部分含义如下。

状态 状态 $s \in S$ 用于刻画当前博弈局面。与 3.1 节中的一般搜索不同，这里的状态除了包含棋盘布局、棋子位置等局面信息，还必须显式包含“当前轮到哪一方行动”这一信息。因为同样的棋盘局面，若轮到不同玩家行动，其战略意义往往完全不同。

初始状态 s_0 表示博弈开始时的状态，它包括初始局面和先手玩家。

动作集合 $A(s)$ 表示在状态 s 下当前玩家可以采取的合法动作集合。例如在井字棋中，动作就是把棋子落在某个空白格中。

状态转移函数

$$s' = \text{Result}(s, a)$$

表示在状态 s 下执行动作 a 后到达的新状态 s' 。

行动玩家函数

$$\text{Player}(s) \in \{\text{MAX}, \text{MIN}\}$$

用于指出在状态 s 下轮到谁行动。通常把试图最大化收益的一方记为 MAX，试图最小化该收益的一方记为 MIN。

终局检测

$$\text{Terminal}(s)$$

用于判断状态 s 是否为终局。例如在井字棋中，一方三子连线或棋盘填满时，游戏结束。

终局效用函数

$$U(s)$$

表示终局状态 s 对 MAX 一方的效用值。由于是零和博弈，若从 MAX 视角的得分为 $U(s)$ ，则从 MIN 视角看，其得分与之相反，因此只需记录一方的效用即可。

为了帮助大家建立直觉，可以先看井字棋这个简单例子。井字棋的棋盘规模很小，规则也较直观，因此非常适合展示“局面展开为搜索树”的过程。一个典型终局如图 3.7 所示。



图 3.7: 井字棋中的一个终局示意图

若把执棋方记为 MAX 和 MIN，则井字棋的终局效用函数通常可以设置为

$$U(s) = \begin{cases} +1, & \text{若 MAX 获胜,} \\ 0, & \text{若平局,} \\ -1, & \text{若 MIN 获胜.} \end{cases}$$

这个定义非常重要，因为后续 Minimax 算法正是通过“把叶结点效用逐层向上回传”的方式，为整棵博弈树中的每个内部结点赋值。

3.3.2 Minimax 搜索算法

基本思想 Minimax 是对抗搜索中最经典、最基础的算法。它的核心思想可以概括为一句话：我方选择能让自己收益尽可能大的动作，同时假设对手也会选择使我方收益尽可能小的动作。于是，搜索树上的不同层会交替出现两种结点：

- ◇ MAX 结点：当前轮到我方行动，应从后继中选择价值最大的分支。
- ◇ MIN 结点：当前轮到对手行动，应从后继中选择价值最小的分支。

这意味着，对抗搜索中的结点价值并不是“当前局面看起来好不好”，而是“在双方都按最优方式行动时，这个局面最终会导致怎样的结果”。因此，Minimax 的本质是一种递归定义的“向后归纳”过程：先计算叶结点效用，再逐层向上决定父结点价值。

Minimax 搜索树的基本结构如图 3.8 所示。

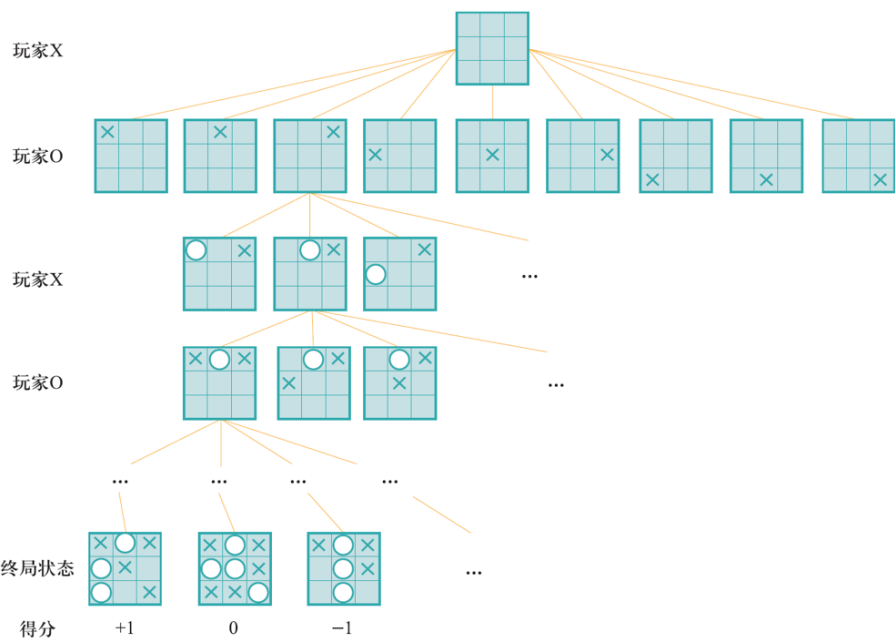


图 3.8: Minimax 搜索树示意图

递推定义 设 $V(s)$ 表示状态 s 在双方最优博弈条件下对 MAX 的价值, 则有如下递推关系:

$$V(s) = \begin{cases} U(s), & \text{若 Terminal}(s) = \text{true} \\ \max_{a \in A(s)} V(\text{Result}(s, a)), & \text{若 Player}(s) = \text{MAX} \\ \min_{a \in A(s)} V(\text{Result}(s, a)), & \text{若 Player}(s) = \text{MIN} \end{cases} \quad (3.12)$$

式 (3.12) 是本节最核心的数学表达。它说明:

第一, 若当前结点已经是终局, 则它的价值就是终局效用。

第二, 若当前轮到 MAX 行动, 则该结点的价值等于所有后继结点价值中的最大值, 因为 MAX 会主动选择对自己最有利的动作。

第三, 若当前轮到 MIN 行动, 则该结点的价值等于所有后继结点价值中的最小值, 因为 MIN 会尽可能压低 MAX 的收益。

于是, 根结点上的最优动作可表示为

$$a^* = \arg \max_{a \in A(s_0)} V(\text{Result}(s_0, a))$$

这里默认初始状态由 MAX 行动。若初始状态由 MIN 行动, 则对应地改为取最小值即可。

一个两层搜索树的直观例子 假设根结点 A 是 MAX 结点, 它有三个后继结点 B, C, D , 而 B, C, D 都是 MIN 结点。若叶结点效用分别如下:

$$B : \{3, 5, 7\}, \quad C : \{2, 9, 4\}, \quad D : \{6, 1, 8\}$$

由于 B, C, D 都由 MIN 决策, 因此

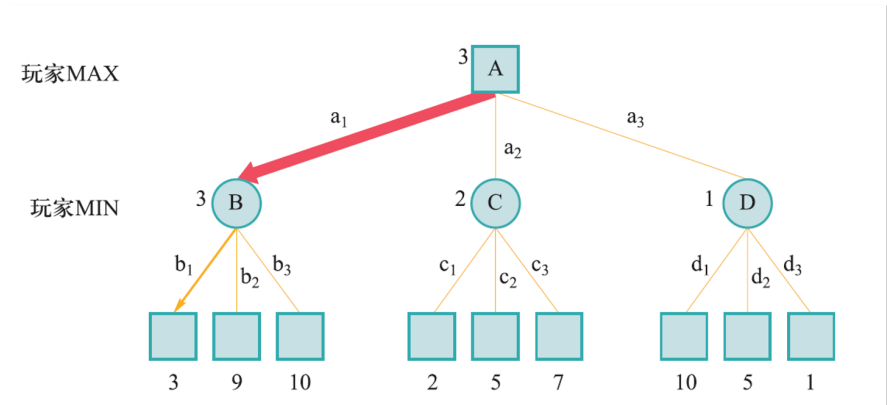
$$V(B) = \min\{3, 5, 7\} = 3, \quad V(C) = \min\{2, 9, 4\} = 2, \quad V(D) = \min\{6, 1, 8\} = 1$$

再向上一层, 由于 A 是 MAX 结点, 所以

$$V(A) = \max\{V(B), V(C), V(D)\} = \max\{3, 2, 1\} = 3$$

上述两层搜索树的价值回传过程如图 3.9 所示。

这说明, 当双方都理性行动时, 根结点的最优价值为 3, MAX 会选择通向 B 的动作。这个例子虽然简单, 却把 Minimax 的“先对手最小化, 再我方最大化”的回传机制表现得很清楚。



Minimax 算法伪代码 Minimax 搜索算法的伪代码如表 3.13 所示。

表 3.13: Minimax 搜索算法伪代码

步骤	伪代码说明
1	定义递归函数 $\text{MinimaxValue}(s)$ 。
2	若 $\text{Terminal}(s) = \text{true}$ ，返回 $U(s)$ 。
3	若 $\text{Player}(s) = \text{MAX}$ ，初始化 $v \leftarrow -\infty$ 。
4	对每个 $a \in A(s)$ ，计算后继状态 $s' = \text{Result}(s, a)$ ，并更新 $v \leftarrow \max\{v, \text{MinimaxValue}(s')\}$ 。
5	若 $\text{Player}(s) = \text{MIN}$ ，初始化 $v \leftarrow +\infty$ 。
6	对每个 $a \in A(s)$ ，计算后继状态 $s' = \text{Result}(s, a)$ ，并更新 $v \leftarrow \min\{v, \text{MinimaxValue}(s')\}$ 。
7	返回 v 。
8	在根结点 s_0 处，枚举每个合法动作 a ，选择使 $V(\text{Result}(s_0, a))$ 最大的动作作为最终决策。

正确性与复杂性分析 Minimax 的正确性来自它对“理性对手”的显式建模。由于每一层都假设当前行动方会采取对自己最有利的动作，因此当递归一直展开到终局时，根结点得到的价值恰好就是双方都采取最优策略时 MAX 能保证获得的结果。换句话说，Minimax 求得的是一种保守而稳健的最优决策。

设博弈树的平均分支因子为 b ，搜索深度为 m ，则在最坏情况下，Minimax 需要展开整棵深度为 m 的搜索树，因此时间复杂度为

$$O(b^m)$$

若采用深度优先方式递归实现，则其空间复杂度通常写为

$$O(bm)$$

若程序按需生成孩子结点，则也常近似写作 $O(m)$ 。从教学角度看，保留 $O(bm)$ 更便于与前文深度优先搜索的分析保持一致。

Minimax 的主要问题在于：当 b 和 m 稍大时，搜索规模会急剧膨胀。例如国际象棋平均每一步可能有几十种合法走法，若向后搜索若干层，结点数就会达到非常惊人的量级。因此，虽然 Minimax 在理论上十分清晰，在实践中却往往需要借助剪枝技术减少搜索量，这就引出了 Alpha Beta 剪枝。

3.3.3 Alpha Beta 剪枝搜索算法

为什么需要剪枝 Minimax 的一个特点是“先算完整棵树，再从叶子向上回传价值”。这种做法在小规模博弈中没有问题，但在复杂博弈中会造成大量无效计算。原因在于：有些分支即使继续展开，也已经不可能改变祖先结点的最终决策了。既然它们对最后结果没有影响，那么继续搜索这些分支就是浪费。

Alpha Beta 剪枝的核心思想正是：在不改变 Minimax 最终结果的前提下，尽早发现“已经不可能影响最终决策”的分支，并停止对这些分支的扩展。它本质上是一种有依据的提前终止机制。

Alpha 与 Beta 的含义 为了理解剪枝条件，先定义两个量：

- ◇ α 表示沿当前搜索路径，MAX 一方已经知道自己至少能够获得的下界。
- ◇ β 表示沿当前搜索路径，MIN 一方已经知道自己至多会允许 MAX 获得的上界。

直观地说， α 是“我方已经争取到的最好保底值”， β 是“对手仍然允许的最高上限”。若在某一个结点上出现

$$\alpha \geq \beta$$

就意味着当前分支的后续搜索已经不可能改变祖先结点的选择，因此可以直接剪枝。

从直观例子理解剪枝 设某个 MAX 结点已经通过前面考察过的兄弟分支知道：自己至少可以获得 3 分，也就是当前 $\alpha = 3$ 。现在继续考察另一个 MIN 子结点。若在这个 MIN 结点的部分后继中，已经发现它最多只能给 MAX 返回 2 分，也就是当前 $\beta = 2$ ，那么剩余后继就没有继续计算的必要了。这一情形可由图 3.10 直观表示。因为对 MIN 来说，它本来就会选择较小值，而 2 已经不高于 MAX 在别处分支中能保证得到的 3 分，所以这整个子树不可能成为根结点的最优选择。

同理，在 MIN 结点的上层，若某个 MAX 后代已经能给出一个不小于当前 β 的值，那么该后代剩余分支也可以停止搜索。图 3.11 给出了这一剪枝情形。图 3.12 为 Alpha Beta 剪枝示意图。

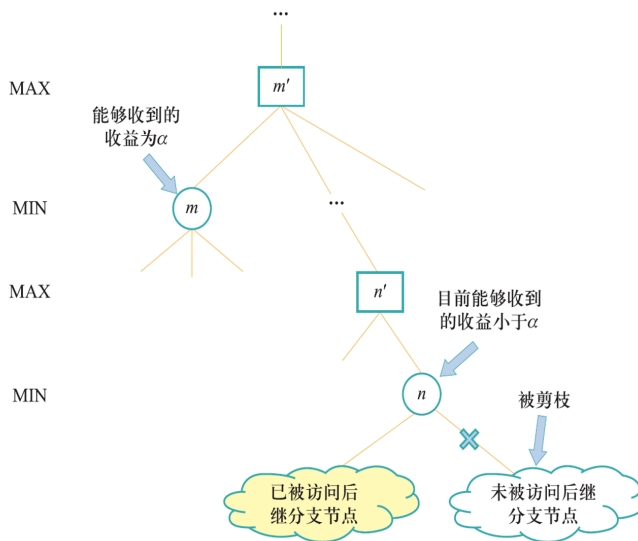


图 3.10: Beta 剪枝示意图

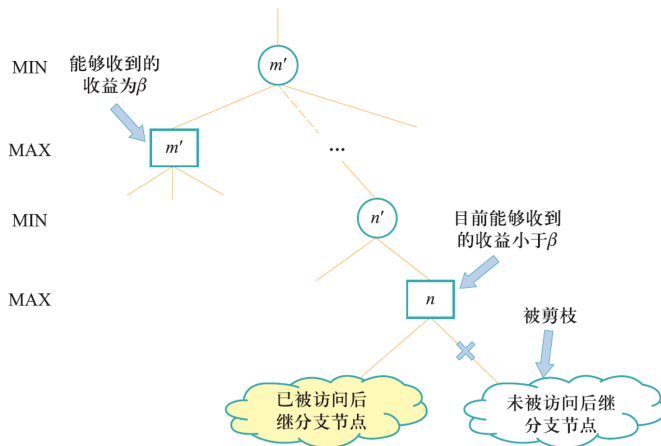


图 3.11: Alpha 剪枝示意图

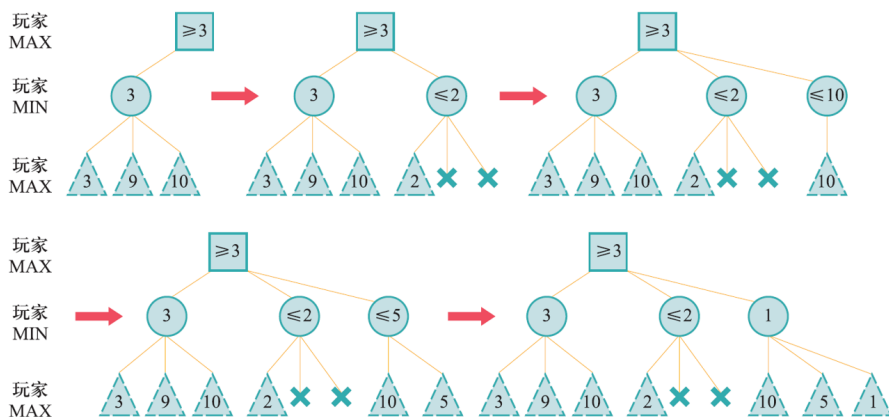


图 3.12: Alpha Beta 剪枝示意图

递归更新规则 Alpha Beta 搜索仍然遵循 Minimax 的极大极小思想，只是在递归过程中多维护了两个边界值。对 MAX 结点而言，若某个孩子返回值为 v ，则更新

$$\alpha \leftarrow \max(\alpha, v)$$

对 MIN 结点而言，若某个孩子返回值为 v ，则更新

$$\beta \leftarrow \min(\beta, v)$$

一旦某结点满足

$$\alpha \geq \beta$$

该结点尚未访问的后继结点便可全部剪去。

从“区间约束”的视角看， α 和 β 实际上是在不断压缩当前结点可能取值的范围。随着搜索深入，这个区间会越来越窄；一旦上下界发生交叉，说明结果已经确定到足以停止继续搜索。图 3.13 展示了区间逐步收缩并触发剪枝的过程。

数学表达 相应地，其数学形式也可以写成如下递归过程：

$$AB(s, \alpha, \beta) = \begin{cases} U(s), & \text{Terminal}(s) = \text{true}, \\ \max_{a \in A(s)} AB(\text{Result}(s, a), \alpha, \beta), & \text{Player}(s) = \text{MAX}, \\ \min_{a \in A(s)} AB(\text{Result}(s, a), \alpha, \beta), & \text{Player}(s) = \text{MIN}, \end{cases}$$

只是实际实现时，每访问一个孩子都要及时更新 α 或 β ，并检查是否满足剪枝条件。

表 3.14: Alpha Beta 剪枝算法伪代码

步骤	伪代码说明
1	定义递归函数 $\text{AlphaBeta}(s, \alpha, \beta)$ 。
2	若 $\text{Terminal}(s) = \text{true}$, 返回 $U(s)$ 。
3	若 $\text{Player}(s) = \text{MAX}$, 初始化 $v \leftarrow -\infty$ 。
4	依次考察每个 $a \in A(s)$, 令 $s' = \text{Result}(s, a)$ 。
5	更新 $v \leftarrow \max\{v, \text{AlphaBeta}(s', \alpha, \beta)\}$ 。
6	更新 $\alpha \leftarrow \max(\alpha, v)$ 。若 $\alpha \geq \beta$, 停止并剪去剩余孩子。
7	若 $\text{Player}(s) = \text{MIN}$, 初始化 $v \leftarrow +\infty$ 。
8	依次考察每个 $a \in A(s)$, 令 $s' = \text{Result}(s, a)$ 。
9	更新 $v \leftarrow \min\{v, \text{AlphaBeta}(s', \alpha, \beta)\}$ 。
10	更新 $\beta \leftarrow \min(\beta, v)$ 。若 $\alpha \geq \beta$, 停止并剪去剩余孩子。
11	返回 v 。根结点初始调用时可令 $\alpha = -\infty, \beta = +\infty$ 。

正确性与性能分析 Alpha Beta 剪枝并没有改变 Minimax 的取值规则, 它只是在搜索过程中省略了那些“已经确定不会被选中”的分支。因此, 它得到的最终决策与完整 Minimax 完全一致。

在最坏情况下, 若结点展开顺序很差, 几乎无法发生有效剪枝, 则 Alpha Beta 的时间复杂度仍然接近

$$O(b^m)$$

然而在理想情况下, 若每一层都优先扩展最优动作, 则搜索规模可显著下降, 其时间复杂度可近似写为

$$O(b^{m/2})$$

这意味着, 在同样计算预算下, Alpha Beta 可以把可搜索深度大致提高一倍左右。虽然这是理想上界, 但它揭示了一个极其重要的事实: 剪枝效果与“结点访问顺序”高度相关。也正因为如此, 现代博弈程序通常会尽量先扩展看起来更有希望的动作, 以便更早触发剪枝。

空间复杂度方面, Alpha Beta 与深度优先形式的 Minimax 基本一致, 通常仍写为

$$O(bm)$$

二者的差异可进一步概括为表 3.15。

表 3.15: Minimax 与 Alpha Beta 的比较

比较维度	Minimax	Alpha Beta
最终决策质量	最优	与 Minimax 相同
是否改变理论结果	否	否
是否减少搜索结点	否	是
最坏时间复杂度	$O(b^m)$	$O(b^m)$
理想时间复杂度	$O(b^m)$	$O(b^{m/2})$
适用场景	小规模博弈或教学示例	中大规模博弈搜索

3.3.4 深度受限搜索与静态估值函数 在井字棋这类小型博弈中，Minimax 可以一直搜索到终局；然而在国际象棋、围棋等复杂博弈中，完全展开到终局通常是不现实的。因此，实际系统往往不会把搜索树扩展到真正结束，而是在某个预设深度处提前停止，并用一个静态估值函数来评估当前局面的好坏。

设截断深度为 d_c ，静态估值函数为 $E(s)$ ，则深度受限的 Minimax 可以写为

$$V_d(s) = \begin{cases} U(s), & \text{若Terminal}(s) = \text{true} \\ E(s), & \text{若} d = d_c \\ \max_{a \in A(s)} V_{d+1}(\text{Result}(s, a)), & \text{若Player}(s) = \text{MAX} \\ \min_{a \in A(s)} V_{d+1}(\text{Result}(s, a)), & \text{若Player}(s) = \text{MIN} \end{cases}$$

这里的 $E(s)$ 用来代替真实终局效用。它不要求完全正确，却应尽量满足一个原则：局面对 MAX 越有利，估值越大；局面对 MIN 越有利，估值越小。以棋类任务为例，估值函数常常综合考虑棋子数量、位置优势、活动空间、王的安全性等因素。

这个思想与 3.2 节中的启发函数有相通之处。启发函数用于估计“距离目标还有多远”，静态估值函数用于估计“当前局面对谁更有利”。两者都不是问题真实解本身，却都能帮助算法在有限计算资源下作出更合理的判断。

3.3.5 本节小结 对抗搜索把前面章节中的一般搜索问题扩展到了多智能体竞争环境中。在这一框架下，结点价值不再由单条路径代价决定，而由双方后续最优决策共同决定。Minimax 通过极大结点与极小结点交替回传价值，给出了零和博弈中的基本求解方法；Alpha Beta 剪枝则进一步说明，很多分支虽然存在，却没有必要真的去搜索，因为它们已经不可能影响最终决策。

从更广的视角看，Minimax 与 Alpha Beta 建立了“博弈树搜索”的经典范式，而 3.4 节的蒙特卡洛树搜索则会在此基础上进一步放宽精确展开的要求，通过随机采样和统计估计，把计算资源优先投入更有希望的分支，从而在更复杂的决策空间中实现高

效求解。

3.4 蒙特卡洛树搜索

在很多智能决策问题中，计算机都要面对一个共同困难，就是“可选方案太多”。例如，下棋时每一步都可能有许多走法，机器人规划路径时也可能面临大量分支。如果把所有可能情况一次性全部展开，再逐个比较，计算量会迅速增大，搜索树很快就会变得十分庞大。

为了解决这个问题，人们提出了蒙特卡洛树搜索方法。所谓“蒙特卡洛”，强调的是随机采样与统计估计；所谓“树搜索”，强调的是问题可以表示成一棵由状态和动作构成的搜索树。蒙特卡洛树搜索的核心思想很直观，就是先有选择地试探一些分支，再根据试探结果把更多计算资源分配给更有希望的分支。随着试探次数不断增加，算法对“哪条路更值得继续走”会越来越有把握。

从直观上看，蒙特卡洛树搜索并不追求一开始就完全算清楚整棵树，而是采用“边试边学、边学边搜”的方式逐步逼近较优决策。它特别适用于搜索空间很大、很难穷举的问题，因此在棋类博弈、路径规划、组合优化等领域都有重要应用。

3.4.1 探索与利用机制的平衡 理解蒙特卡洛树搜索，首先要理解“探索”和“利用”这一对核心概念。

先看一个生活中的例子。假设一名学生中午去食堂吃饭，面前有五个窗口。前几天他试过其中三个，感觉4号窗口最好，菜品稳定，口味也不错。这时他面临一个选择：今天继续去4号窗口，还是尝试以前没怎么去过的5号窗口。

继续选择4号窗口，属于“利用”。因为已有经验表明它表现不错，继续选它通常比较稳妥。尝试5号窗口，属于“探索”。因为虽然对它了解不多，但它也许比4号窗口更好。

如果这个学生每次都只去自己当前最喜欢的窗口，那么他也许会长期停留在一个“还不错但并非最好”的选择上；如果他每天都完全随机地换窗口，那么他又无法充分利用已经积累起来的经验。可见，在许多智能决策问题中，系统都必须在探索与利用之间找到一个合适的平衡点。

这个思想放到搜索树中也完全一样。设想一个棋类程序站在根结点处，面前有多个候选走法。某些走法已经被试探过很多次，看起来效果较好；另一些走法暂时访问较少，真实潜力仍然不清楚。若程序只盯着当前统计最好的分支，它可能错过更优选择；若它平均地试探每个分支，又会浪费大量计算资源。因此，搜索算法必须回答一个关键问题：在下一步扩展时，究竟应该优先走向“当前看起来最好”的分支，还是“尚未充分了解”的分支。

蒙特卡洛树搜索给出的答案是：同时考虑这两个因素，用一个统一的评分公式来平衡“历史表现”和“信息不足”所带来的价值。这个评分思想，正是从多臂赌博机问题中的上置信区间方法发展而来的。

3.4.2 上置信区间方法 为了更正式地描述“探索与利用”的平衡，可以先从多臂赌博机问题出发。设有若干个可选动作，每次选择一个动作后都会得到一个随机奖励。算法的目标是在不断试探中逐渐识别出平均奖励更高的动作。

设第 i 个动作在第 j 次被选择时得到的奖励为 $X_{i,j}$ ，到第 t 轮为止，该动作被选择的次数记为 $N_i(t)$ 。于是，第 i 个动作在时刻 t 的经验平均奖励可表示为式 (3.13)

$$\bar{X}_i(t) = \frac{1}{N_i(t)} \sum_{j=1}^{N_i(t)} X_{i,j} \quad (3.13)$$

若仅根据经验平均奖励 $\bar{X}_i(t)$ 来决策，算法会反复选择当前均值最高的动作。这种策略实现简单，却容易过早相信“暂时领先”的动作，因为样本少时的平均值常常带有较大随机波动。

为此，上置信区间方法在经验均值之外再加入一个探索奖励项，得到

$$\text{UCB}_i(t) = \bar{X}_i(t) + C \sqrt{\frac{\ln t}{N_i(t)}} \quad (3.14)$$

其中， $C > 0$ 为控制探索强度的常数。

式 (3.14) 由两部分组成。第一部分 $\bar{X}_i(t)$ 反映动作 i 的历史平均表现，数值越大，说明该动作在过去的尝试中表现越好，因此更值得继续利用。第二部分 $C \sqrt{\frac{\ln t}{N_i(t)}}$ 则反映该动作当前仍有多大“不确定性”。当某个动作的尝试次数 $N_i(t)$ 很小时，这一项较大，说明它还没有被充分了解，因此应继续给予试探机会；当 $N_i(t)$ 持续增大时，这一项会逐渐减小，说明该动作的统计估计已经越来越稳定。

从趋势上看，式 (3.14) 体现了两个重要性质。其一，随着总尝试轮数 t 增加， $\ln t$ 会缓慢增长，因此算法始终保留一定探索意愿。其二，随着某个动作被访问次数 $N_i(t)$ 增加，分母变大，探索奖励项会减小，因此算法会逐步把更多注意力放到真正表现较好的动作上。

因此，UCB 的核心思想可以概括为：历史平均表现好的动作值得继续利用，访问次数较少的动作值得继续探索，而算法通过一个统一公式在二者之间取得平衡。

为了更直观地理解这一点，考虑一个简单例子。假设某时刻共有三个候选动作 A, B, C ，父结点已被访问 $N = 39$ 次，取 $C = \sqrt{2}$ 。三个动作的统计信息如表 3.16 所示。

从表 3.16 可以看到，动作 C 的历史平均收益最低，但由于它只被尝试过一次，探索奖励很大，因此当前仍值得继续试探。这个例子说明，UCB 关注的并不只是“谁当前均值最高”，还包括“谁最值得再看一眼”。

表 3.16: UCT 评分示例

动作	累计收益 W	访问次数 n	平均收益 W/n	UCT 值
A	18	30	0.600	$0.600 + \sqrt{2}\sqrt{\ln 39/30} \approx 1.094$
B	6	8	0.750	$0.750 + \sqrt{2}\sqrt{\ln 39/8} \approx 1.707$
C	0	1	0.000	$0 + \sqrt{2}\sqrt{\ln 39/1} \approx 2.707$

3.4.3 从 UCB 到 UCT 多臂赌博机讨论的是多个动作之间的选择问题，而树搜索中需要解决的是：在搜索树的某个结点处，下一步应优先沿着哪一个子结点继续向下搜索。两者的表面形式不同，核心矛盾却一致，都是在“看起来表现较好的对象”和“尚未被充分了解的对象”之间做平衡。

因此，人们将 UCB 的思想推广到树结构中，得到 UCT 方法。对于某个父结点 v 的子结点 v_j ，其评分通常定义为

$$\text{UCT}(v_j) = \bar{Q}(v_j) + C\sqrt{\frac{\ln N(v)}{N(v_j)}} \quad (3.15)$$

其中

$$\bar{Q}(v_j) = \frac{W(v_j)}{N(v_j)} \quad (3.16)$$

这里， $W(v_j)$ 表示结点 v_j 的累计收益， $N(v_j)$ 表示该结点的访问次数， $N(v)$ 表示其父结点的访问次数。由式 (3.16) 可知， $\bar{Q}(v_j)$ 就是该子结点当前的平均收益估计。

式 (3.15) 与式 (3.14) 在形式上几乎完全一致。差别只在于，前者应用于树结构中的结点选择，后者应用于若干独立动作之间的选择。于是，UCT 实际上给出了一个非常清晰的规则：在选择阶段，总是优先进入当前 UCT 值最大的那个子结点。

这个规则有非常明确的直观意义。若某个子结点平均收益高，那么式 (3.15) 的第一项较大，它会因为历史表现较好而更容易被继续利用；若某个子结点访问次数少，那么第二项较大，它会因为仍存在较大不确定性而更容易被继续探索。这样一来，算法既不会只盯着少数早期领先的分支，也不会毫无方向地平均试探所有分支，而是会把更多计算资源逐步集中到更有希望的区域。

需要指出的是，常数 C 的取值会明显影响搜索风格。若 C 取得较小，算法会更重视当前平均收益，整体风格偏向利用；若 C 取得较大，算法会更愿意访问次数较少的分支，整体风格偏向探索。因此， C 的本质作用就是调节探索与利用之间的平衡程度。

3.4.4 蒙特卡洛树搜索的四个基本阶段 有了 UCT 选择规则之后，蒙特卡洛树搜索的整体流程就可以清晰地表示出来。一次完整的 MCTS 迭代通常由四个阶段组成，即选择、扩展、模拟和回传。

选择 选择阶段从根结点出发，沿着当前已经建立的搜索树不断向下。在每一个已展开结点处，算法根据 UCT 规则，从其子结点中选出一个当前最值得继续探索的结点。这个过程会持续进行，直到遇到以下两种情况之一：

第一，到达一个尚未完全展开的结点；

第二，到达一个终局结点。

所谓“尚未完全展开”，是指该结点仍然存在合法动作对应的后继状态尚未加入搜索树。若到达这种结点，搜索将进入扩展阶段。

扩展 若选择阶段停在一个非终局且尚未完全展开的结点上，算法就从该结点尚未尝试过的合法动作中选一个，将对应的新结点加入树中。扩展阶段的作用在于逐步生长搜索树，使得原本只存在于理论状态空间中的后继状态，逐渐转化为真正被搜索和统计过的树结点。

可以把扩展理解为“在当前搜索树边界上打开一个新分支”。没有扩展，算法只能在已有树结构中反复游走；有了扩展，搜索树才会随着搜索进行而不断向外生长。

模拟 扩展出新结点后，算法通常不会立刻对这个新结点继续进行复杂的深入搜索，而是从该结点出发，按照某种较快的策略一直走到终局，或者走到预先给定的截断深度后使用评估函数估计当前局面价值。这个过程称为模拟，也常称为 rollout。

在最基础的 MCTS 中，模拟常采用随机策略，也就是在后续每一步中随机选择一个合法动作，直到到达终局。设本次模拟最终得到的终局收益为 z ，则 z 就是这一次试探对当前分支给出的样本评价。

对于二人零和博弈，常将 z 取为式 (3.17)

$$z \in \{1, 0, -1\} \quad (3.17)$$

分别表示根结点行动方最终获胜、平局和失败。对于更一般的问题，奖励也可以是任意实数值，例如路径总代价、资源调度回报等经过适当变换后的收益。

模拟阶段的意义在于，用大量相对便宜的快速试探替代对整棵子树的完全穷举。单次模拟结果可能带有较强随机性，但随着模拟次数增加，这些随机样本会逐渐累积出更稳定的统计趋势。

回传 当一次模拟结束后，算法会把本次得到的结果 z 沿着刚才经过的路径逐层向上传递，并更新路径上各结点的统计量。最常见的更新方式为式 (3.18)、式 (3.19) 和式 (3.20)

$$N(v) \leftarrow N(v) + 1 \quad (3.18)$$

$$W(v) \leftarrow W(v) + z \quad (3.19)$$

$$\bar{Q}(v) = \frac{W(v)}{N(v)} \quad (3.20)$$

其中， $N(v)$ 表示结点访问次数， $W(v)$ 表示累计收益， $\bar{Q}(v)$ 表示平均收益估计。

在教学中，一个容易混淆的问题是“收益应按谁的视角记录”。常见做法有两种。第一种做法是始终从根结点行动方的视角记录收益，这样整条路径上的所有结点都回传同一个 z 。第二种做法是按当前行动方的视角记录收益，此时由于相邻层代表的行动方不同，在回传时常需进行符号翻转。对于本科教学，采用第一种方式通常更直观，也更便于大家理解。

因此，一次完整的 MCTS 迭代可以概括为：从根结点出发，根据 UCT 规则向下选择；到达边界后扩展一个新结点；从新结点出发进行快速模拟；再将模拟结果沿路径回传并更新统计量。随着这一过程不断重复，算法对各个分支优劣的认识会越来越清晰。

3.4.5 蒙特卡洛树搜索的伪代码 为了把四个阶段与具体执行流程对应起来，可以将 MCTS 概括为表 3.17 所示的伪代码：

伪代码虽然不长，却非常清晰地体现了 MCTS 的基本思想：搜索树并不是一开始就完整生成的，而是在“边试探边更新”的循环中逐步生长起来的。每一轮搜索都只扩展一小部分树结构，但长期积累后，统计信息会越来越丰富，决策也会越来越稳定。

在实际程序实现中，还常会加入若干细节。例如，对尚未访问过的子结点，可将其 UCT 值直接视为一个很大的数，以保证每个动作至少会被尝试一次；在模拟阶段，也可采用“随机策略加启发规则”的混合方式，以提高样本质量；在最终输出动作时，很多系统选择根结点访问次数最多的动作，而不是平均收益最高的动作，因为访问次数往往代表了更稳定的统计证据。

表 3.17: 蒙特卡洛树搜索伪代码

蒙特卡洛树搜索**输入:** 根结点 $root$, 搜索预算 $Budget$ **输出:** 根结点处推荐动作 $best_action$

```

for iter = 1 to Budget do
    node  $\leftarrow$  root
    # 1. 选择
    while node 为非终局结点且 node 已完全展开 do
        node  $\leftarrow$  argmax_child UCT(child)
    end while
    # 2. 扩展
    if node 为非终局结点且 node 未完全展开 then
        从 node 的未尝试动作中选一个动作  $a$ 
        child  $\leftarrow$  Expand(node,  $a$ )
        node  $\leftarrow$  child
    end if
    # 3. 模拟
     $z \leftarrow$  Rollout(node)
    # 4. 回传
    while node  $\neq$  null do
         $N(node) \leftarrow N(node) + 1$ 
         $W(node) \leftarrow W(node) + z$ 
        node  $\leftarrow$  Parent(node)
    end while
end for

```

返回根结点处访问次数最多的动作

3.4.6 一个简化的博弈例子 为了帮助大家理解“为什么 MCTS 会越搜越准”，下面考虑一个简化的棋类决策场景。设根结点有三个候选动作 a_1, a_2, a_3 。搜索刚开始时，这三个动作都只被试探过很少次数，因此算法对它们的认识都较为粗糙。经过若干轮选择、扩展、模拟和回传之后，可能会得到如表 3.18 所示的统计结果。

表 3.18: 根结点处三个候选动作的统计示例

动作	访问次数 N	累计收益 W	平均收益 W/N	直观解释
a_1	40	24	0.60	表现稳定，长期较好
a_2	12	9	0.75	当前均值更高，但样本偏少
a_3	3	0	0.00	当前表现差，且统计证据不足

如果只比较平均收益，算法会更倾向于选择 a_2 。但 MCTS 并不会仅仅根据这一列数值立即下结论，因为 a_2 的样本较少，还需要继续验证；同时， a_3 虽然当前表现较差，但访问次数很少，理论上仍可能存在被低估的情况。因此，在随后的若干轮搜索中，UCT 会继续为 a_2 和 a_3 提供一定的探索机会。

如果后续搜索表明 a_2 确实更强，那么随着访问次数增加，它的高收益会持续被验证，平均收益会逐步稳定下来，并吸引更多搜索预算；如果 a_2 只是早期运气较好，那么随着样本增多，它的平均收益会逐渐回落，算法也会把更多搜索资源转移给真正更优的分支。

由此可见，MCTS 的优势在于它允许算法在不断试探中修正自己的判断，而不是要求一开始就做出十分准确的结论。这种“先粗略试探，再逐步聚焦”的思想，正是它在大规模搜索问题中表现出色的重要原因。

3.4.7 蒙特卡洛树搜索的特点 从方法性质上看，MCTS 同时具有“树搜索方法”和“统计采样方法”的双重特征。

首先，它是一种树搜索方法。因为算法始终围绕状态结点、动作分支和搜索路径来组织计算，目标是在庞大的状态空间中找到当前最值得执行的动作。

其次，它又是一种统计采样方法。因为算法并不试图精确展开并求值整棵搜索树，而是通过大量模拟得到样本，再利用样本统计量近似估计各结点的价值。

正因为如此，MCTS 具有以下几个突出特点。

第一，它特别适合分支因子较大的问题。在这类问题中，完整展开搜索树的代价往往极高，而 MCTS 可以把有限计算资源优先投入到更有希望的区域。

第二，它是一种 anytime 方法。随着搜索时间增加，算法通常会给出更稳定、更可靠的决策；即便在中途停止，它往往也已经能够给出一个可用答案。

第三，它对高质量启发函数的依赖相对较弱。基础 MCTS 即使只使用随机模拟，也能够工作；若进一步引入更好的策略模型、价值估计模型或领域启发规则，性能还可以继续提升。

第四，它在有限预算下给出的通常是近似最优解，而不是严格意义上的全局最优解。由于模拟本身带有随机性，在预算有限时，统计估计仍可能存在偏差，因此搜索结果本质上是一种逐步逼近式的决策。

3.4.8 蒙特卡洛树搜索与 Minimax 的比较 在学习完 3.3 节的对抗搜索后，大家很自然会提出一个问题：既然已经有 Minimax 搜索和 Alpha Beta 剪枝，为什么还需要 MCTS？

两类方法的根本区别，在于它们使用了不同的求解思想。Minimax 搜索强调对博弈树进行系统推理，尽可能依据递归关系和局面评估得到较准确的结点值；Alpha Beta 剪枝则在不改变最终结论的前提下减少无效搜索。MCTS 则采用采样估计的思想，不试图穷尽整棵树，而是通过大量模拟逐渐判断哪些分支更值得深入。

从适用场景看，若搜索树分支较可控，且评估函数较强，那么 Minimax 及其剪枝技术通常表现很好；若搜索空间非常庞大，完整展开代价过高，那么 MCTS 往往更具优势，因为它可以把资源动态集中到更有希望的分支上。

表 3.19 给出了两者的简单比较。

表 3.19: Minimax 搜索与 MCTS 的对比

比较维度	Minimax 与 Alpha Beta	蒙特卡洛树搜索
基本思想	系统推理，递归求值	随机采样，统计估值
适合场景	分支较可控，评估函数较强	分支较大，难以穷举
决策性质	偏向精确推理	偏向近似估计
时间增加后的表现	搜得更深	样本更多，统计更稳
对评估函数依赖	较强	基础版较弱，增强版可较强

可以用一句形象的话帮助大家区分二者：Minimax 更像是尽力把树“算清楚”，MCTS 更像是不断对树进行“试探和统计”。

3.4.9 现代人工智能中的 MCTS 从更现代的角度看，MCTS 的重要性还在于它非常容易与机器学习方法结合。基础 MCTS 已经能够通过随机模拟进行决策，而当系统拥有更强的策略模型和价值模型后，这些模型又可以帮助 MCTS 更高效地选择分支和评估局面。

例如，在棋类人工智能系统中，策略模型可以为“哪些动作更值得优先扩展”提供先验概率，价值模型可以在未到达终局时直接给出局面价值估计，从而减少纯随机模拟的成本。这样一来，树搜索就形成了“学习模型提供方向，树搜索负责验证与分配计算资源”的协同机制。

这说明，在现代人工智能中，搜索与学习并不是彼此分离的两条路线。相反，二者常常深度融合，共同构成复杂智能系统的决策核心。

3.4.10 本节小结 蒙特卡洛树搜索是在巨大搜索空间中进行近似决策的一种重要方法。它通过随机采样不断积累统计信息，并借助 UCT 机制在探索与利用之间取得平衡。一次完整的 MCTS 迭代包含选择、扩展、模拟和回传四个阶段。随着模拟次数增加，算法会逐步把更多计算资源分配给更有希望的分支，从而不断提高决策质量。

从知识联系上看，3.1 节建立了搜索问题的统一表示框架，3.2 节介绍了启发式信息如何指导搜索，3.3 节讨论了对抗环境下的精确推理，而 3.4 节则进一步展示了：当搜索空间过大、难以完全穷举时，如何利用随机采样和统计估计完成有效决策。至此，搜索算法在人工智能中的几条典型技术路线就形成了较完整的知识链条。

符号说明 为了便于大家复习，本节常用符号总结如表 3.20 所示。

表 3.20: MCTS 常用符号说明

符号	含义
$N(v)$	结点 v 的访问次数
$W(v)$	结点 v 的累计收益
$\bar{Q}(v)$	结点 v 的平均收益, $\bar{Q}(v) = W(v)/N(v)$
$N(v_j)$	子结点 v_j 的访问次数
$N(v)$	父结点 v 的访问次数
C	探索强度常数，用于控制探索与利用的平衡
z	一次模拟返回的终局收益
$\bar{X}_i(t)$	第 i 个动作在时刻 t 的经验平均奖励
$UCB_i(t)$	多臂赌博机中的上置信区间评分
$UCT(v_j)$	树搜索中对子结点 v_j 的综合评分